

Salve a tutti,

in questa sezione verranno analizzati esclusivamente i codici del Prof. Guglielmo De Angelis, forniti durante le lezioni.

Questa parte è di estrema importanza. Non bisogna pensare che i contenuti siano incomprensibili o che non valga la pena affrontarli; al contrario, come vedrete, rivestono un ruolo fondamentale, poiché contribuiscono a sviluppare un corretto modello di ragionamento e a formare la giusta mentalità per affrontare con successo l'esame.

Un buon approccio è quello di seguire tre passaggi:

1. Lettura: comprendere la struttura del codice.
2. Esecuzione: provarlo in un ambiente di sviluppo.
3. Sperimentazione: modificarlo per osservare gli effetti delle variazioni.
Questo metodo vi permetterà non solo di ricordare meglio, ma soprattutto di interiorizzare un vero *modus operandi* da ingegnere del software.

Ogni tentativo di sperimentazione consolida le conoscenze e favorisce la creazione di collegamenti logici.

Best, Simone.

Iniziamo in modo semplice, affrontando il tema dei **costruttori**.

Darò per scontato che il lettore abbia già familiarità con il concetto di costruttore e con il suo ruolo fondamentale nella creazione e nell'inizializzazione degli oggetti. Pertanto, non ci soffermeremo sulla definizione di base, ma andremo a considerare aspetti applicativi ed esempi concreti che ci permetteranno di comprendere meglio il loro utilizzo in fase di progettazione.

```
/*
 *      Copyright (C) Guglielmo De Angelis (a.k.a. Gulyx)
 */

public class A {
    private int attr1;

    //-----
    // Try to swap the comments for these statements
    //-----
    private B rifB;
    //-----
    //      private C rifC;
    //-----
    //-----

    public A() {
        System.out.println("here is A()");
    //-----
    // Try to swap the comments for these statements
    //-----
        this.rifB = new B(this);
    //-----
    //      this.rifC = new C(this);
    //-----
    //-----
        System.out.println("done: A()");
    }

    public void op1() {
        System.out.println("here is A.op1()");
    }

    public static void main(String args[]) {
        System.out.println("***** Instantiation of class
B ***** ");
        B b = new B();
        System.out.println("***** done!! ***** ");
    }
}
```

```

        System.out.println("***** Instantiation of class
C ***** ");
        C c = new C();
        System.out.println("***** done!! ***** ");

        System.out.println("***** Instantiation of class
A ***** ");
        A a = new A();
        System.out.println("***** done!! ***** ");
    }
}

```

```

/*
 *   Copyright (C) Guglielmo De Angelis (a.k.a. Gulyx)
 */

public class B {
/* -----
As final experience, try to remove the all the comments
marked as:
"-- UNCOMMENT THE FOLLOWING STATEMENT --"
in all the classes and run again the example.
----- */

    private int attr2;

    protected A rifA;

    public B() {
        System.out.println("default B");
        System.out.println("done: B()");
    }

    public B(A rifA) {
        System.out.println("here is B(A rifA)");
        this.rifA = rifA;
    }

    // // -----
    // // -- UNCOMMENT THE FOLLOWING STATEMENT --
    //         this.op2();
    // // -----

```

```

        System.out.println("done: B(A rif A)");
    }

    public void op2() {
        System.out.println("here is B.op2()");

        // // -----
        // // -- UNCOMMENT THE FOLLOWING STATEMENT --
        //         rifA.op1();
        // // -----
    }
}

-----
public class C extends B{
/* -----
    As final experience, try to remove the all the comments
    marked as:
    "-- UNCOMMENT THE FOLLOWING STATEMENT --"
    in all the classes and run again the example.
----- */

    public C(){
//         super();
        System.out.println("default C");
        System.out.println("done: C()");
    }

    public C(A rifA){
        super(rifA);
        System.out.println("here is C(A rifA)");
// // -----
// // -- UNCOMMENT THE FOLLOWING STATEMENT --
//         this.rifA.op1();
// // -----
        System.out.println("done: C(A rifA)");
    }

    public void op2() {
        System.out.println("here is C.op2()");
// // -----
// // -- UNCOMMENT THE FOLLOWING STATEMENT --
//         this.rifA.op1();
// // -----
    }
}

```

```

        public void op3() {
            System.out.println("here is C.op2()");
// // -----
// // -- UNCOMMENT THE FOLLOWING STATEMENT --
//         this.rifA.op1();
// // -----
        }
    }

}



---


public class D {
    int foo;

//     public D(){
//         new D(500);
//     }

    public D(){
        this(500);
    }

    public D(int foo){
        this.foo = foo;
    }

    public static void main (String args[]) {
        System.out.println("Attempt to create a new instance
for D ...");

        D d = new D();

        System.out.println("... done!");

        System.out.println("This is the value in foo: " +
d.foo);
    }
}

```

Output del programma:

```

***** Instantiation of class B *****
default B
done: B()
***** done!! *****

```

```

***** Instantiation of class C *****
default B
done: B()
default C
done: C()
***** done!! *****
***** Instantiation of class A *****
here is A()
here is B(A rifA)
done: B(A rif A)
done: A()
***** done!! *****

```

1. Passo.

Su A.java, viene creato il costruttore di B, quando questo viene creato stampa

```
default B
```

```
done: B()
```

```
***** done!! *****
```

La classe C estende B, quindi quando fai `new C()`, prima viene invocato il costruttore della superclasse B, poi quello di C.

Se nel costruttore di una sottoclasse non scrivi esplicitamente `super( ... )`, il compilatore inserisce automaticamente un `super( )` come prima istruzione. Questo `super( )` invoca il costruttore senza parametri della superclasse.

Poi continua con il costruttore `C()` → stampa “default C” e “done: C()”.

2. Passo.

Viene chiamato il costruttore di A.

Dopodiché, A crea un B passandogli "sé stesso": `this.rifB = new B(this);`

Così stampa `here is B(A rifA)` e `done: B(A rif A)`.

e Infine `done: A()` dentro A.

Non so se ci siamo capiti, quello che è successo è davvero banale ma interessante: A chiama il costruttore di B passando sé stesso. Immagina che B sia un buco nero, A ci è andato a finire dentro.

Immagina che **A sia un pianeta**.

Ad un certo punto, A decide di creare un **buco nero B**.

Ma per farlo funzionare, non basta un po' di energia: A deve **gettare sé stesso dentro il buco nero**.

New B(this) è come dire: “Creo un buco nero, ma dentro ci butto me stesso”.

È un legame reciproco e un po' “paradossale”: A esiste, ma una parte di A è anche dentro B.



Adesso analizziamo il codice relativo agli “Scope”. Lo scope è l’ambito di visibilità di una variabile o di un identificatore, cioè dove nel codice quella variabile può essere usata.

Lo scope definisce **fino a dove una variabile è “visibile” e utilizzabile** nel programma.

```
public class A {  
  
    public int attr1;  
    public int attr2;  
  
    public A(){  
        attr1 = 10;  
        attr2 = 20;  
        System.out.println("In A --> attr1 : " + attr1 + "  
attr2 : " + attr2);  
    }  
}
```

```

    }

    public A(int attr1){
        this.attr1 = attr1;
        attr2 = 20;
        System.out.println("In A --> attr1 : " + attr1 + "
attr2 : " + attr2);
    }

    public static void main(String args[]) {
        System.out.println("***** Instantiation of class A
***** ");
        A a = new A();
        System.out.println("***** done!! ***** ");
        System.out.println("***** Instantiation of class B
***** ");
        B b = new B();
        System.out.println("***** done!! ***** ");
        System.out.println("***** Another Instantiation of
class B ***** ");
        B otherb = new B(300,100);
        System.out.println("***** done!! ***** ");
    }
}

```

```

public class B extends A {
    public int attr1;
    public int attr3;

    public B(){
        this.attr1 = 30;
        this.attr3 = 40;
        System.out.println("In B --> attr1 : " + super.attr1
+ " attr3 : " + this.attr3);
    }

    public B(int attr1Local, int attr1Super){

```

```

        super(attr1Super);
        this.attr1 = attr1Local;
        this.attr3 = 40;
        System.out.println("In B --> attr1 : " + super.attr1
+ " attr3 : " + this.attr3);
    }

    public void op1(){

    }

}

```

Output del programma:

```

***** Instantiation of class A *****
In A --> attr1 : 10 attr2 : 20
***** done!! *****
***** Instantiation of class B *****
In A --> attr1 : 10 attr2 : 20
In B --> attr1 : 10 attr3 : 40
***** done!! *****
***** Another Instantiation of class B *****
In A --> attr1 : 100 attr2 : 20
In B --> attr1 : 100 attr3 : 40
***** done!! *****

```

Questo mi sembra abbastanza semplice no?

Il Professore, in questo esempio, vuole farci capire questo:

B ridefinisce `attr1`, che non è lo stesso `attr1` di A.

- `super.attr1` → prende quello di A.
- `this.attr1` → prende quello di B.

